Cell-Based Architecture

make every outage a small outage

Watch the 4-min video

In one line: split the system into independent cells, each a complete stack serving a subset of customers, behind a thin router, so a bug, overload or bad deploy hits one cell's customers instead of all of them; shuffle sharding shrinks the blast radius of a single bad customer even further.

1 Everyone down at once

A bad configuration change reaches the service and every customer goes down together. Now imagine the same bug hits **5%** of customers and everyone else never notices. That is the promise of cells: they limit the **blast radius**, the share of customers one failure can reach (lesson 7's bulkheads, applied to the whole system).

2 What a cell is

A **cell** is a complete, independent copy of the service stack: its own load balancer, app servers, database and queues. Each serves a subset of customers. Cells **share no state** and **don't call each other**. If one fails, from a bug, a poison request, overload or a bad deploy, only its customers are affected.

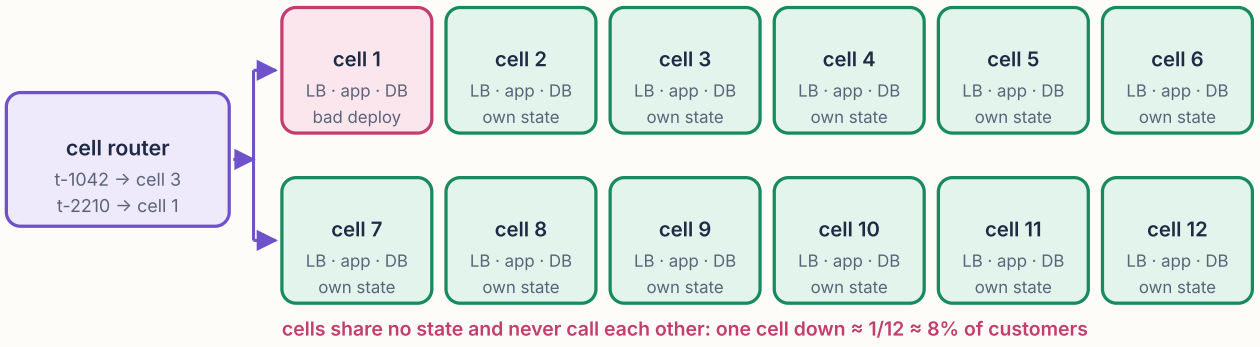


Figure 1. Twelve cells behind a router that maps each account to its cell.

Cells have a **fixed maximum size**, so you can load-test exactly that size, and you grow by **adding cells**, not by making one huge system bigger. The **cell router** in front only looks up the customer's cell (by account or tenant ID in a mapping table) and forwards the request. It is shared by everyone, so it must be very simple and very reliable: no business logic, and cached mappings so it keeps routing if the mapping store is down (lesson 18).

3 Safer deploys, and noisy neighbours

- **Staged rollout by cell:** deploy to one cell, watch its alarms, then a few more, then the rest. A bad change is caught while it affects one cell (lesson 75).
- **Isolation of big tenants:** move a noisy customer into its own cell so it can't hurt anyone else.
- **Migration** between cells is needed for rebalancing; design for it from the start.

4 Shuffle sharding

Instead of fixed groups, give each customer a **random combination** of workers, say 2 of 8. There are C(8,2) = 28 possible pairs, so two customers rarely share exactly the same pair. If one customer sends poison requests that crash its workers, other customers almost always keep at least one healthy worker. AWS describes using this technique for Route 53's name servers.

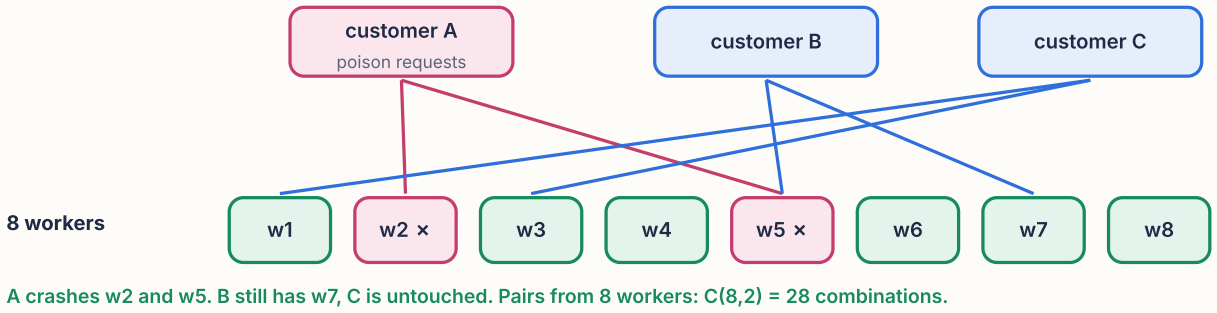


Figure 2. Customer A's poison requests take out its two workers. Only a customer with exactly the same pair would lose all its capacity.

Layout	One bad deploy reaches	One poison customer reaches
One shared fleet	100% of customers	100% of customers
12 cells	1 cell first: ≈ 8%	its cell: ≈ 8%
Shuffle shards (2 of 8)	depends on rollout	fully: only customers on the same pair (1 in 28 ≈ 3.6%); partly: those sharing one worker

WORKED EXAMPLE: TWELVE CELLS AND A CACHE BUG

A service runs as **12 cells**, each with its own load balancer, compute and database, sized for a few thousand customers. A cell router maps each account to a cell. A deploy contains a bug that corrupts a cache. It reaches **cell 1** first, alarms fire, and the pipeline rolls back automatically. About **8%** of customers (1 of 12) saw errors for a few minutes; the other 92% saw nothing, and the bug never reached the other cells.

TRADE-OFFS

- More infrastructure and some duplicated fixed cost per cell.
- Operations across many cells: deploys, dashboards and migrations must be automated.
- Cross-customer features (global search, reports) don't fit naturally.
- The router and the cell mapping become a critical shared component.

5 Key terms

Cell: complete, independent stack serving a subset of customers

Cell router: thin layer mapping each customer to a cell

Shuffle sharding: random combination of workers per customer

Blast radius: share of customers one failure reaches

Partition key: what decides the cell: account or tenant ID

Poison request: input that crashes whatever processes it

6 Interview questions

Q1. What is a cell? BASIC

A complete, independent copy of the service stack (load balancer, compute, data) serving a subset of customers, sharing no state with other cells.

Q2. Why must the cell router be simple? BASIC

Every request passes through it, so it is the one shared component; any complexity or dependency in it can take all cells down.

Q3. How does cell-based architecture limit outages? SCENARIO

A bug, overload or bad deploy affects only one cell's customers, and staged rollouts cell by cell catch problems while only one cell is exposed.

Q4. One huge tenant keeps overloading its cell. What do you do? SCENARIO

Move it to a dedicated cell (or a larger cell class) so its load can't affect the others.

Q5. What is shuffle sharding? TRICKY

Giving each customer a random combination of workers, so one bad customer's failure rarely takes out all of another customer's capacity.

Q6. Why give cells a fixed maximum size? TRICKY

You can test that exact size thoroughly and avoid the unknown scaling limits of one ever-growing system; growth means adding cells.

Next: SD-K-017 · Chaos Engineering backendroadmap.com